



Architetture moderne per il calcolo parallelo

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

Nelle puntate precedenti....

- Le prestazioni **non crescono più automaticamente** con l'hardware
 - La fine dello scaling di Dennard ha reso il **parallelismo inevitabile**
 - Il modello sequenziale classico **non describe** l'hardware moderno
 - I sistemi di calcolo attuali sono:
 - multicore
 - gerarchici (memoria)
 - paralleli a più livelli
 - L'HPC nasce dall'integrazione di:
 - architettura
 - software
 - algoritmi
-

Obiettivi della lezione

Al termine della lezione lo studente sarà in grado di:

- Comprendere **perché l'architettura hardware influenza le prestazioni**
 - Riconoscere i principali **componenti di un processore moderno**
 - Capire il ruolo della **gerarchia di memoria** e del principio di località
 - Distinguere tra:
 - parallelismo a livello di istruzioni
 - parallelismo sui dati
 - parallelismo a livello di thread
 - Inquadrare le architetture moderne nel contesto dell'**HPC**
-

Perché parlare di architettura in un corso di programmazione parallela

Le prestazioni di un programma parallelo dipendono dall'**architettura hardware**

Lo stesso codice può avere prestazioni **molto diverse** su architetture diverse

L'architettura determina

- costi di accesso alla memoria
- disponibilità di parallelismo
- costo della comunicazione e sincronizzazione

Programmare senza conoscere l'architettura porta a

- scelte inefficienti
- scarsa scalabilità

La programmazione parallela efficace richiede **consapevolezza dell'hardware sottostante**

Dal modello ideale alla realtà

Il modello ideale di calcolo

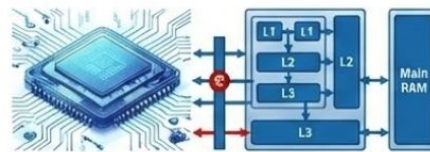


- esecuzione sequenziale
- costo accesso alla memoria uniforme
- costo trascurabile delle operazioni

Questo modello è utile per:

- descrivere algoritmi
- ragionare sulla correttezza

L'hardware reale è invece



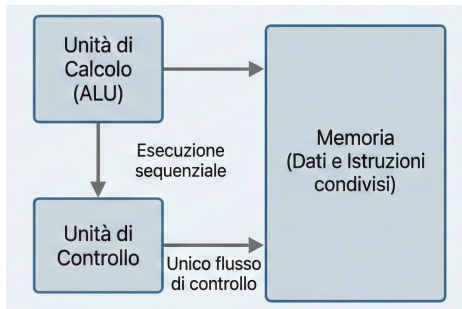
- parallelo
- gerarchico (memoria)
- non uniforme nei costi

Le prestazioni reali dipendono da:

- località dei dati
- parallelismo sfruttato
- interazione con l'architettura

Modello di riferimento: Von Neumann (e perché non basta)

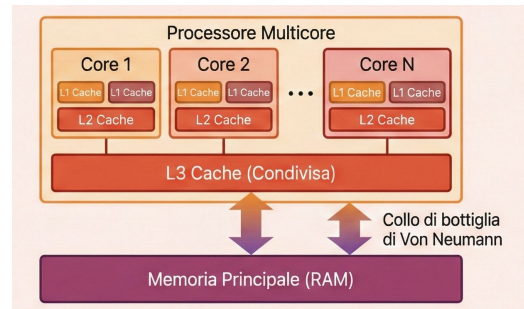
Il modello di Von Neumann



- Componenti: Unità di Calcolo, Memoria, Unità di controllo
- Spazio di memoria condiviso per istruzioni e dati
- Modello di esecuzione sequenziale delle istruzioni
- Unico flusso di controllo

Alla base dei linguaggi di programmazione e del modello sequenziale

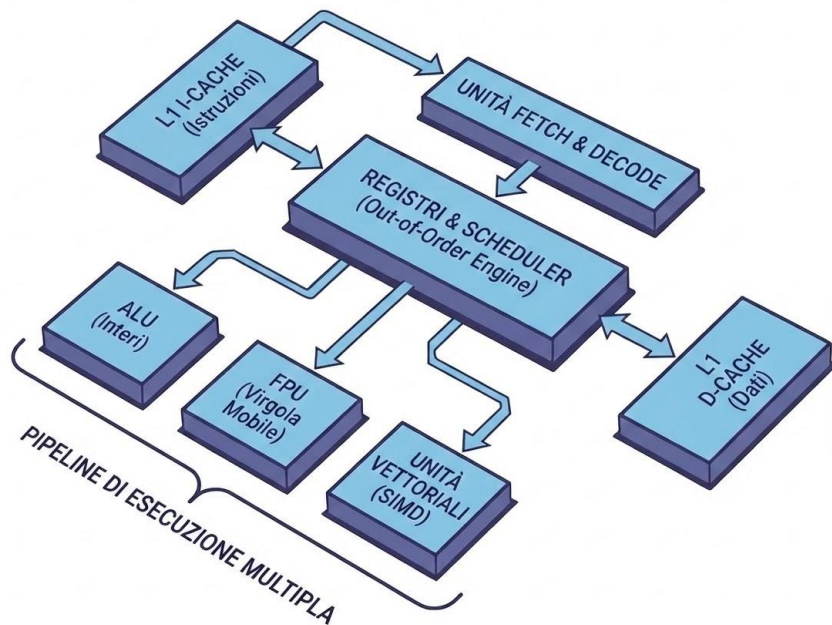
Perché non basta (i limiti moderni)



- Non descrive: Parallelismo interno (ILP, multicore)
- Gerarchia di Memoria (Registri, Cache, RAM, storage)
- Costi reali di accesso ai dati (latenza, larghezza di banda)

Necessario un modello più complesso per prestazioni ed efficienza moderne

Anatomia di un core moderno



Componenti specializzati

- Unità **Fetch/Decode**
- **Pipeline** di esecuzione
- **Unità di esecuzione** (ALU, FPU, unità vettoriali)
- **Registri**
- **Cache di primo livello (L1)**

Sfruttare il parallelismo interno

- Obiettivo: Aumentare il Throughput
- Esecuzione Fuori Ordine: Esegue istruzioni non appena i dati sono pronti, non in sequenza
- Sovrapposizione (Pipelining): Più stadi attivi contemporaneamente

Il problema centrale: la Memory Wall

- La **Memory Wall** descrive il divario crescente tra:
 - velocità di calcolo della CPU
 - velocità di accesso alla memoria principale
- Negli ultimi decenni:
 - le CPU sono diventate **molto più veloci**
 - la latenza della memoria è migliorata **molto più lentamente**
- Il risultato è che:
 - il processore spesso **rimane inattivo** in attesa dei dati
 - le prestazioni sono limitate dall'accesso alla memoria, non dal calcolo
- La Memory Wall è uno dei principali **colli di bottiglia** nei sistemi moderni



Gran parte dell'architettura moderna è progettata per mitigare questo problema

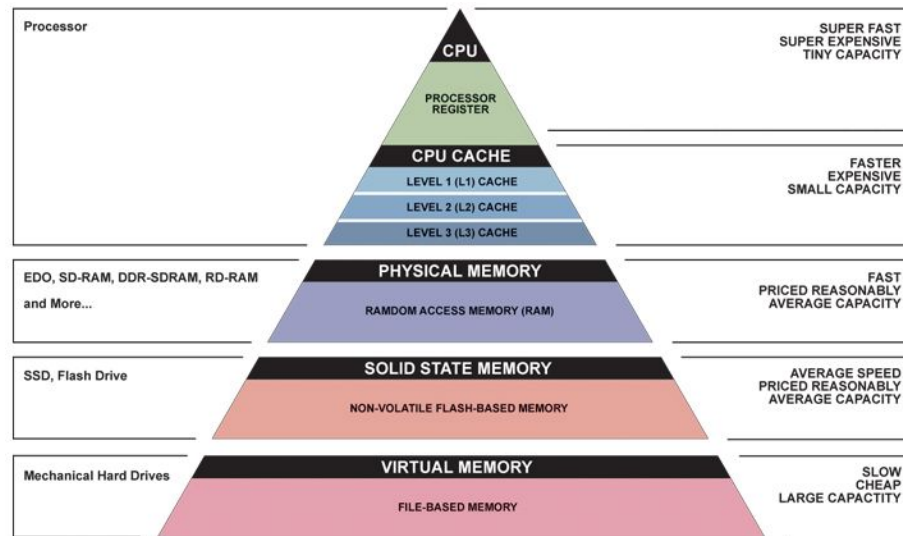
Dal memory wall alla soluzione: il principio di località

- Il **principio di località** descrive il modo in cui i programmi accedono ai dati
- Due forme principali di località:
 - **Località temporale**: dati usati di recente tendono a essere riutilizzati
 - **Località spaziale**: dati vicini in memoria tendono a essere usati insieme
- L'architettura moderna sfrutta la località tramite:
 - cache
 - prefetching
 - gerarchia di memoria
- Le prestazioni dipendono da **quanto bene il codice sfrutta la località**

Scrivere codice che viola la località peggiora drasticamente le prestazioni

Gerarchia di memoria

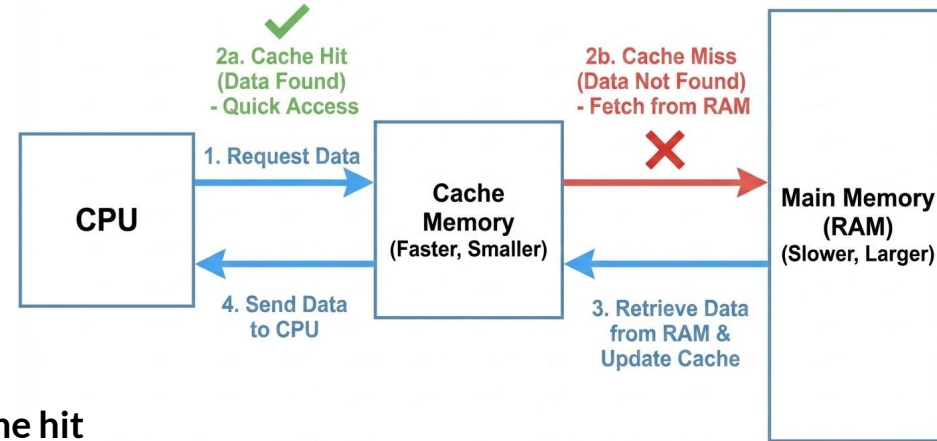
- I sistemi moderni utilizzano una **gerarchia di memoria** con più livelli
- Ogni livello differisce per:
 - **latenza**
 - **banda**
 - **capacità**
- Tipici livelli della gerarchia:
 - **Registri** (più veloci, molto pochi)
 - **Cache L1** (per core)
 - **Cache L2** (per core o condivisa)
 - **Cache L3** (condivisa tra core)
 - **Memoria principale (RAM)**
- Più la memoria è lontana dalla CPU:
 - maggiore è la latenza
 - minore è la velocità
- Le prestazioni dipendono da **dove si trovano i dati**



▲ Simplified Computer Memory Hierarchy
Illustration: Ryan J. Leng

Cache: concetti fondamentali

- Le **cache** sono memorie veloci che memorizzano copie di dati presenti in memoria principale
- I dati sono trasferiti tra memoria e cache in **blocchi** (cache line)
- Una cache è caratterizzata da:
 - **dimensione**
 - **latenza**
 - **associatività**
- Quando un dato è presente in cache si ha un **cache hit**
- Quando un dato non è presente in cache si ha un **cache miss**
- I cache miss comportano un **costo elevato** in termini di tempo
- L'efficacia delle cache dipende dal **principio di località**



Prefetching e accesso ai dati

- Il **prefetching** è un meccanismo che anticipa il caricamento dei dati in cache
- Può essere:
 - **hardware** (automatico, basato su pattern di accesso)
 - **software** (esplicito, tramite istruzioni o suggerimenti)
- Il prefetching è efficace quando:
 - gli accessi ai dati sono **regolari**
 - i pattern sono **prevedibili**
- Accessi irregolari o indiretti:
 - riducono l'efficacia del prefetching
 - aumentano i cache miss

Le prestazioni dipendono dalla regolarità dell'accesso ai dati

Prefetching e accesso ai dati

- Il **prefetching** è un meccanismo che anticipa il caricamento dei dati in cache
- Può essere:
 - **hardware** (automatico, basato su pattern di accesso)
 - **software** (esplicito, tramite istruzioni o suggerimenti)
- Il prefetching è efficace quando:
 - gli accessi ai dati sono **regolari**
 - i pattern sono **prevedibili**
- Accessi irregolari o indiretti:
 - riducono l'efficacia del prefetching
 - aumentano i cache miss

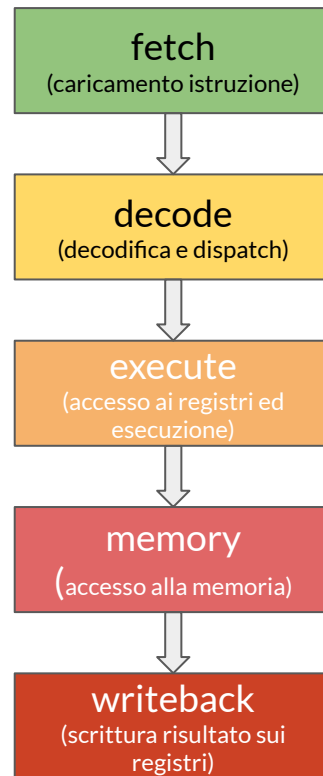
```
for (i = 0; i < N; i++)  
    sum += A[i];
```

```
for (i = 0; i < N; i++)  
    sum += A[index[i]];
```

Le prestazioni dipendono dalla regolarità dell'accesso ai dati

Pipeline

- La **pipeline** suddivide l'esecuzione di un'istruzione in **più stadi**
- Tipici stadi della pipeline:
 - fetch
 - decode
 - execute
 - memory access
 - write-back
- Più istruzioni possono essere **in esecuzione contemporaneamente** in stadi diversi
- L'obiettivo della pipeline è aumentare il **throughput**, non ridurre la latenza
- La pipeline introduce:
 - dipendenze tra istruzioni
 - possibili stalli (*pipeline stalls*)



Instruction-Level Parallelism (ILP)

- L'**Instruction-Level Parallelism (ILP)** è il parallelismo tra istruzioni **indipendenti**
 - Un processore moderno può:
 - eseguire più istruzioni nello stesso ciclo di clock
 - riordinare dinamicamente le istruzioni (*out-of-order execution*)
 - L'ILP è sfruttato tramite:
 - pipeline multiple
 - unità di esecuzione multiple
 - Il grado di ILP è limitato da:
 - dipendenze tra istruzioni
 - accessi alla memoria
 - struttura del codice
 - L'ILP è **trasparente al programmatore**, ma ha limiti intrinseci
-

Instruction-Level Parallelism (ILP)

- L'**Instruction-Level Parallelism (ILP)** è il parallelismo tra istruzioni **indipendenti**
- Un processore moderno può:
 - eseguire più istruzioni nello stesso ciclo di clock
 - riordinare dinamicamente le istruzioni (*out-of-order execution*)
- L'ILP è sfruttato tramite:
 - pipeline multiple
 - unità di esecuzione multiple
- Il grado di ILP è limitato da:
 - dipendenze tra istruzioni
 - accessi alla memoria
 - struttura del codice
- L'ILP è **trasparente al programmatore**, ma ha limiti intrinseci

Caso 1

```
a = b + c;  
d = a * e;  
f = d - g;
```

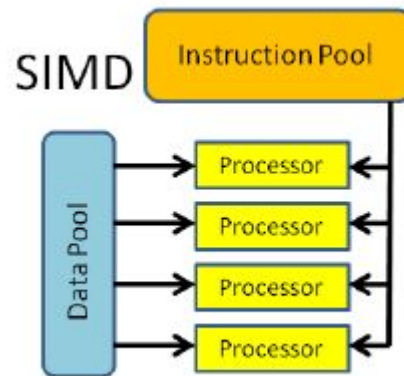
Caso 2

```
a = b + c;  
d = e * f;  
g = h - i;
```

SIMD e vettorizzazione

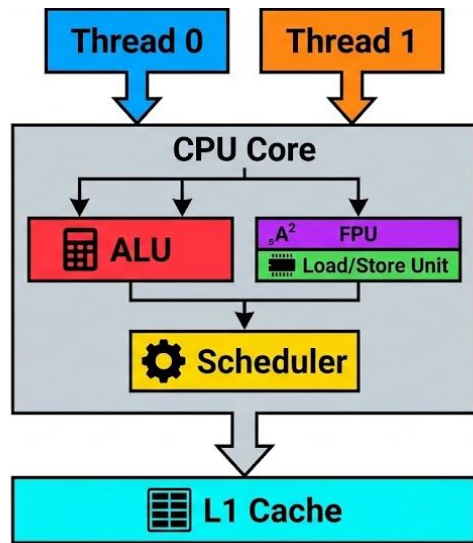
- **SIMD (Single Instruction, Multiple Data)** permette di applicare la stessa operazione a **più dati contemporaneamente**
- Le CPU moderne includono **unità vettoriali** (es. AVX) con registri di grande ampiezza
- La **vettorizzazione** può essere:
 - automatica (a carico del compilatore)
 - esplicita (a carico del programmatore)
- La velocità ottenibile dipende da:
 - regolarità degli accessi alla memoria
 - assenza di dipendenze tra iterazioni

La vettorizzazione è molto efficace, ma non sempre applicabile



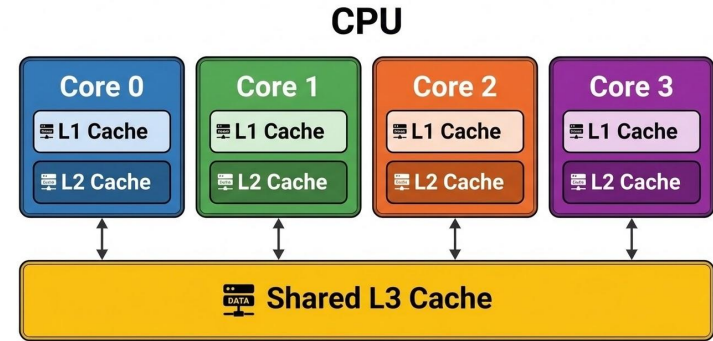
Multithreading hardware (SMT)

- Lo **SMT (Simultaneous Multithreading)** permette a un core di eseguire più thread hardware contemporaneamente
- I thread condividono:
 - pipeline
 - unità di esecuzione
 - cache
- Lo SMT mira a:
 - migliorare l'utilizzo delle risorse del core
 - nascondere latenze (es. accessi in memoria)
- SMT **non raddoppia** le prestazioni:
 - i thread competono per le stesse risorse
- L'efficacia dello SMT dipende:
 - dal tipo di carico di lavoro
 - dal comportamento di memoria



Multicore processors

- I **processori multicore** integrano più **core di calcolo** sullo stesso chip
- Ogni core dispone di:
 - pipeline e unità di esecuzione proprie
 - cache private (tipicamente L1 e L2)
- Alcune risorse sono **condivise** tra i core:
 - cache di ultimo livello (L3)
 - memoria principale
- Il multicore è la principale risposta hardware alla:
 - fine dello scaling di Dennard
 - impossibilità di aumentare la frequenza
- Le prestazioni dipendono dalla capacità del software di:
 - sfruttare più core
 - gestire la condivisione delle risorse



MIMD

Definizione (Classificazione di Flynn)

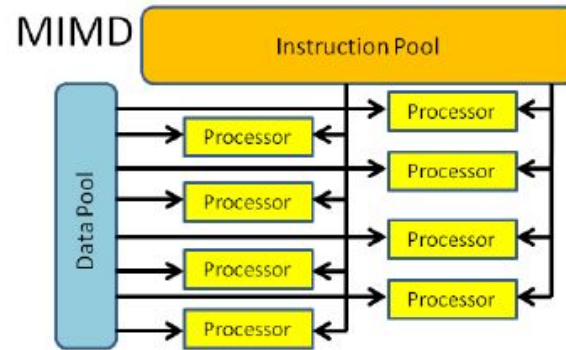
- Ogni unità di elaborazione esegue un **flusso di istruzioni indipendente**
- Ogni unità opera su **dati differenti**
- Parallelismo coarse-grained

Dove si trova nei sistemi moderni

- CPU multicore
- Sistemi SMP
- Architetture NUMA
- Cluster HPC (tra nodi)
- Programmazione MPI
- Thread OpenMP su core diversi

Caratteristiche principali

- Elevata flessibilità
- Adatto a problemi eterogenei
- Comunicazione esplicita o tramite memoria condivisa
- Scalabilità potenzialmente elevata



Symmetric Multiprocessing (SMP)

Definizione

è un'architettura in cui:

- più processori condividono la **stessa memoria fisica**
- lo **stesso spazio di indirizzamento**
- tutti i processori hanno pari accesso alle risorse
- il sistema operativo vede i processori come equivalenti

Vantaggi

- Modello concettualmente semplice
- Comunicazione veloce intra-nodo
- Supporto diretto da OS e runtime

Caratteristiche principali

- Modello **UMA (Uniform Memory Access)**
- Coerenza di cache gestita dall'hardware
- Comunicazione tra thread tramite memoria condivisa
- Programmazione tipicamente a thread (es. OpenMP)

Limiti

- Scalabilità limitata
 - Contesa sul bus/memoria
 - Problemi di coerenza e traffico cache
 - Difficoltà a scalare oltre poche decine di core
-

NUMA e ccNUMA

- **NUMA (Non-Uniform Memory Access)** indica un'architettura in cui:
 - la memoria è **fisicamente distribuita**
 - il tempo di accesso dipende dalla **posizione** dei dati
- Ogni gruppo di core è associato a una porzione di memoria **locale**
- L'accesso a memoria:
 - **locale** → più veloce
 - **remota** → più lento
- **ccNUMA (cache-coherent NUMA):**
 - mantiene la **coerenza delle cache** tra i nodi NUMA
 - presenta una **memoria condivisa dal punto di vista del programmatore**

Le prestazioni dipendono dalla **località NUMA** dei dati e dei thread

Sistemi a memoria condivisa

- In un **sistema a memoria condivisa** tutti i thread:
 - vedono lo **stesso spazio di indirizzamento**
 - possono accedere agli stessi dati
 - Il parallelismo è espresso tramite:
 - **thread**
 - Vantaggi:
 - programmazione più semplice
 - comunicazione implicita tramite memoria
 - Svantaggi:
 - competizione per le risorse
 - problemi di sincronizzazione
 - costi di accesso alla memoria **non uniformi** (NUMA)
-

Sistemi a memoria distribuita

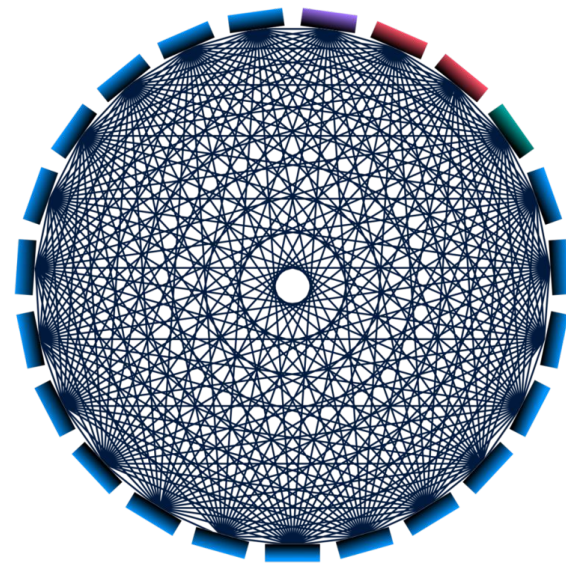
- In un **sistema a memoria distribuita**:
 - ogni processo ha una **memoria privata**
 - non esiste uno spazio di indirizzamento globale
 - I nodi di un cluster HPC sono tipicamente:
 - sistemi a memoria distribuita
 - La comunicazione tra processi avviene tramite:
 - **scambio esplicito di messaggi**
 - Vantaggi:
 - elevata **scalabilità**
 - nessuna contesa diretta sulla memoria
 - Svantaggi:
 - programmazione più complessa
 - costo esplicito della comunicazione
-

Sistemi ibridi (cluster di nodi multicore)

- I sistemi HPC moderni sono **architetture ibride**
 - Ogni sistema è composto da:
 - **nodi multicore** a memoria condivisa
 - collegati in un **cluster a memoria distribuita**
 - All'interno del nodo:
 - parallelismo tramite **thread**
 - memoria condivisa (spesso **NUMA**)
 - Tra nodi:
 - parallelismo tramite **processi**
 - comunicazione esplicita
 - Questo modello combina:
 - flessibilità
 - prestazioni
 - scalabilità
-

Interconnessioni di rete

- Nei sistemi HPC i nodi sono collegati tramite **reti ad alte prestazioni**
- La rete influenza direttamente:
 - **latenza** della comunicazione
 - **banda** disponibile
 - **scalabilità** delle applicazioni
- Le reti HPC sono progettate per:
 - bassa latenza
 - alta banda
 - comunicazioni parallele
- Esempi di tecnologie:
 - InfiniBand
 - Omni-Path
 - Ethernet ad alte prestazioni
- La comunicazione di rete è spesso il **fattore limitante** nella scalabilità



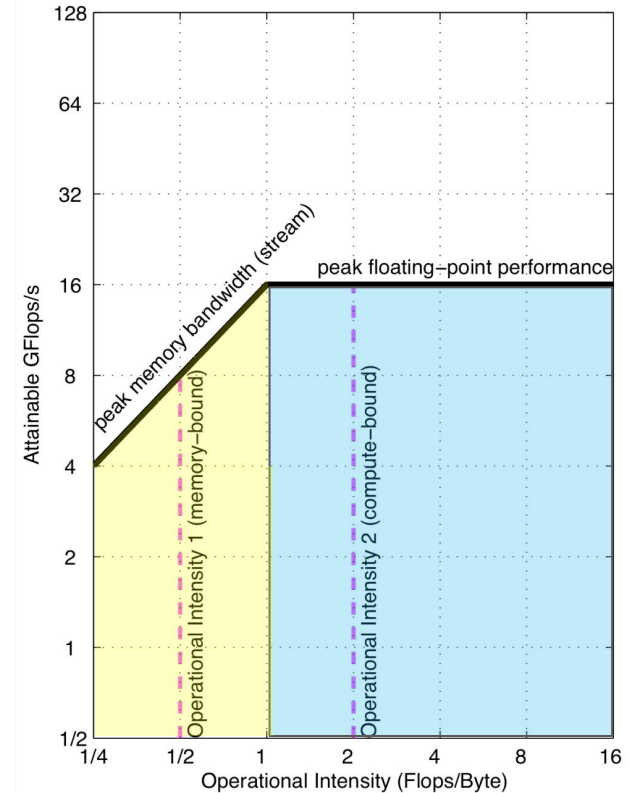
Booster Module nodes
I/O cell
Data-Centric cells
Hybrid cell (Booster + Data-Centric nodes)

Acceleratori

- Gli **acceleratori** sono dispositivi specializzati per il calcolo parallelo
 - L'esempio più comune è la **GPU**
 - Caratteristiche principali:
 - elevato numero di unità di calcolo
 - forte parallelismo sui dati
 - elevata banda di memoria
 - Gli acceleratori sono particolarmente efficaci per:
 - calcolo numerico intensivo
 - operazioni regolari su grandi insiemi di dati
 - L'uso degli acceleratori richiede:
 - modelli di programmazione specifici
 - gestione esplicita dei dati
 - CPU e acceleratori lavorano in modo **complementare**
-

Roofline model

- Il **Roofline model** è un modello prestazionale semplificato
- Descrive il limite massimo di prestazioni di un'applicazione
- Le prestazioni sono limitate da:
 - **capacità di calcolo** (peak FLOPS)
 - **banda di memoria**
- Il modello mette in relazione:
 - **intensità computazionale** (operazioni / byte)
 - **prestazioni ottenibili**
- Esistono due regimi principali:
 - **memory-bound**
 - **compute-bound**
- Il Roofline aiuta a capire **dove intervenire per ottimizzare**



Implicazioni per il programmatore HPC

- Le prestazioni non dipendono solo dall'algoritmo, ma dalla sua **interazione con l'architettura**
 - Il programmatore deve:
 - pensare in termini di **parallelismo**
 - considerare la **gerarchia di memoria**
 - sfruttare la **località dei dati**
 - Non esiste un'unica soluzione ottimale:
 - le scelte dipendono dall'hardware
 - Ottimizzare significa:
 - ridurre accessi costosi alla memoria
 - aumentare il lavoro utile per dato
 - Misurare e analizzare è fondamentale
-

Podcast

Questo podcast è stato generato con strumenti di Intelligenza Artificiale (<https://notebooklm.google/>) utilizzando contenuti e fonti forniti dall'autore del corso.

Il materiale ha esclusivamente funzione di supporto e sintesi degli argomenti trattati a lezione. Non sostituisce il materiale didattico ufficiale, le slide, i testi di riferimento o lo studio individuale.

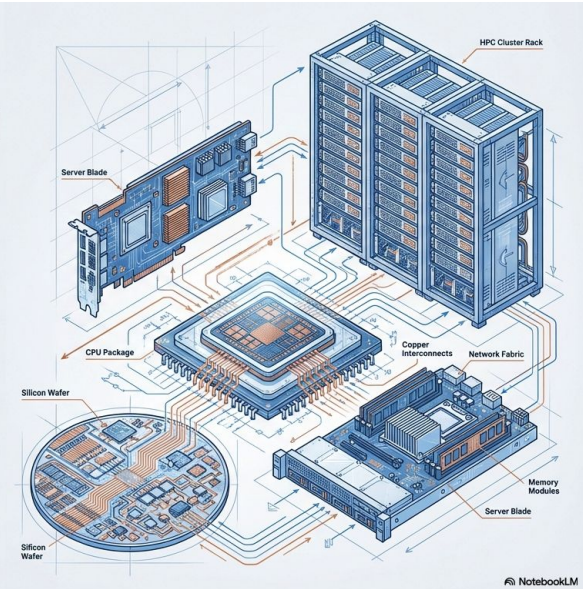
Il podcast deve essere inteso come uno strumento integrativo per il ripasso e il consolidamento dei concetti, e non come fonte esaustiva o alternativa allo studio approfondito.

Architetture Moderne per l'HPC: Dal Silicio al Supercomputer

Perché capire l'hardware è l'unico modo per scrivere software efficiente

Il calcolo ad alte prestazioni (HPC) nasce dall'integrazione di architettura, modelli di programmazione e algoritmi. Non possiamo ottimizzare ciò che non comprendiamo fisicamente.

- **Il Micro:** Core, Pipeline, SIMD
- **L'Ostacolo:** The Memory Wall
- **Il Macro:** Multicore, NUMA, Cluster



<https://youtu.be/EBoMYFFJHR8>